

Computational Methods Project: Numerical Solutions of First-Order ODEs

Daniel Li

[Code on Github Repo](#)

Introduction

Analytical solutions to ODEs cannot always be found, so numerical methods like Euler's Method and Heun's Method can help approximate a solution by stepping forward in time and simulating the slope. The accuracy of these methods then depend on the step size h .

Euler's Method

Given a step size h and an initial value $y(t_0) = y_0$, it advances the solution by approximating the derivative as constant over each interval:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

Each step extrapolates linearly along the tangent to the solution curve at (t_n, y_n) . Euler's method is only first-order accurate. The global error scales as $O(h)$.

Heun's Method

Heun's method is a second-order explicit method that corrects Euler's tendency to drift by incorporating a predictor-corrector approach. It first takes a full Euler step to obtain a predicted value $\tilde{y}_{n+1}$, then averages the slopes at both endpoints:

$$\tilde{y}_{n+1} = y_n + hf(t_n, y_n)$$

$$y_{n+1} = y_n + \frac{h}{2}[f(t_n, y_n) + f(t_{n+1}, \tilde{y}_{n+1})]$$

By averaging the slope at the beginning and end of each interval, Heun's method achieves second-order accuracy. The global error scales as $O(h^2)$. This makes it significantly more accurate than Euler's method for the same step size h .

Assigned ODEs

4. $y' - 2y = \sin(0.5t)$, $[0, 4\pi]$, $y(0) = 0$

8. $y' + y = 3e^{-t}$, $[0, 1]$, $y(0) = 1$

12. $y' + y = t^2 + 2t + 1$, $[0, 1]$, $y(0) = 1$

Part A: Analytical Solutions

ODE 4

ODE: $y' - 2y = \sin(\frac{t}{2})$

Interval: $[0, 4\pi]$

Initial Value: $y(0) = 0$

Homogeneous

$$\begin{aligned} y'_c - 2y_c &= 0, y_c = ce^{rt} \rightarrow r - 2 = 0 \\ &\rightarrow y_c = ce^{2t} \end{aligned}$$

Particular

$$\begin{aligned}
y_p &= A \cos\left(\frac{t}{2}\right) + B \sin\left(\frac{t}{2}\right) \\
\rightarrow y_p' &= -\frac{A}{2} \sin\left(\frac{t}{2}\right) + \frac{B}{2} \cos\left(\frac{t}{2}\right) \\
y_p' - 2y_p &= -\frac{A}{2} \sin\left(\frac{t}{2}\right) + \frac{B}{2} \cos\left(\frac{t}{2}\right) - 2A \cos\left(\frac{t}{2}\right) - 2B \sin\left(\frac{t}{2}\right) = \sin\left(\frac{t}{2}\right) \\
\rightarrow -\frac{A}{2} - 2B &= 1, \frac{B}{2} - 2A = 0 \rightarrow A = -\frac{2}{17}, B = -\frac{8}{17} \\
\rightarrow y_p &= -\frac{2}{17} \cos\left(\frac{t}{2}\right) - \frac{8}{17} \sin\left(\frac{t}{2}\right)
\end{aligned}$$

General Solution

$$y = y_c + y_p = ce^{2t} - \frac{2}{17} \cos\left(\frac{t}{2}\right) - \frac{8}{17} \sin\left(\frac{t}{2}\right)$$

Apply Initial Value

$$\begin{aligned}
y(0) &= c - \frac{2}{17} = 0 \rightarrow c = \frac{2}{17} \\
\rightarrow y &= \frac{2}{17}e^{2t} - \frac{2}{17} \cos\left(\frac{t}{2}\right) - \frac{8}{17} \sin\left(\frac{t}{2}\right)
\end{aligned}$$

$$\text{Explicit Analytical Solution: } y = \frac{2}{17}e^{2t} - \frac{2}{17} \cos\left(\frac{t}{2}\right) - \frac{8}{17} \sin\left(\frac{t}{2}\right)$$

ODE 8

$$\text{ODE: } y' + y = 3e^{-t}$$

$$\text{Interval: } [0, 1]$$

$$\text{Initial Value: } y(0) = 1$$

Homogeneous

$$\begin{aligned}
y_c' + y_c &= 0, y_c = ce^{rt} \rightarrow r + 1 = 0 \\
\rightarrow y_c &= ce^{-t}
\end{aligned}$$

Particular

$$\begin{aligned}
y_p &= Ate^{-t} \\
\rightarrow y_p' &= Ae^{-t} - Ate^{-t} \\
y_p' + y_p &= Ae^{-t} - Ate^{-t} + Ate^{-t} = Ae^{-t} = 3e^{-t} \rightarrow A = 3 \\
\rightarrow y_p &= 3te^{-t}
\end{aligned}$$

General Solution

$$y = y_c + y_p = ce^{-t} + 3te^{-t}$$

Apply Initial Value

$$y(0) = c + 0 = 1 \rightarrow y = e^{-t} + 3te^{-t}$$

$$\text{Explicit Analytical Solution: } y = e^{-t} + 3te^{-t}$$

ODE 12

$$\text{ODE: } y' + y = t^2 + 2t + 1$$

$$\text{Interval: } [0, 1]$$

$$\text{Initial Value: } y(0) = 1$$

Homogeneous

$$\begin{aligned}
y_c' + y_c &= 0, y_c = ce^{rt} \rightarrow r + 1 = 0 \\
\rightarrow y_c &= ce^{-t}
\end{aligned}$$

Particular

$$\begin{aligned}
y_p &= At^2 + Bt + C \\
\rightarrow y_p' &= 2At + B
\end{aligned}$$

$$y_p' + y_p = At^2 + (2A + B)t + (B + C) = t^2 + 2t + 1 \rightarrow A = 1, B = 0, C = 1$$

$$\rightarrow y_p = t^2 + 1$$

General Solution

$$y = y_c + y_p = ce^{-t} + t^2 + 1$$

Apply Initial Value

$$y(0) = c + 1 = 1 \rightarrow y = t^2 + 1$$

Explicit Analytical Solution: $y = t^2 + 1$

Code

Imports

```
import numpy as np
import matplotlib.pyplot as plt
from collections.abc import Callable
f64 = np.float64
f64arr = np.typing.NDArray[f64]
```

Plotting Functions

```
def approx_sol_euler(yp: Callable[[f64, f64], f64], t: f64arr, y0: f64) -> f64arr:
    y = np.zeros(t.size, dtype=f64)
    y[0] = y0

    for i in range(1, y.size):
        y[i] = y[i-1] + (t[i] - t[i-1]) * yp(t[i-1], y[i-1])
    return y

def approx_sol_heun(yp: Callable[[f64, f64], f64], t: f64arr, y0: f64) -> f64arr:
    y = np.zeros(t.size, dtype=f64)
    y[0] = y0

    for i in range(1, y.size):
        pred = y[i-1] + (t[i] - t[i-1]) * yp(t[i-1], y[i-1])
        y[i] = y[i-1] + (t[i] - t[i-1])/2 * (yp(t[i-1], y[i-1]) + yp(t[i], pred))
    return y

def approx_sol(yp: Callable[[f64, f64], f64], t0: f64, L: f64, y0: f64):
    divs = [1, 2, 4, 8, 16]
    colors_euler = ["#f6bdc0", "#f1959b", "#f07470", "#ea4c46", "#dc1c13"]
    colors_heun = ["#bfbfff", "#a3a3ff", "#7879ff", "#4949ff", "#1f1fff"]
    for div, col_e, col_h in zip(divs, colors_euler, colors_heun):
        t = np.linspace(t0, t0 + L, div*10 + 1)
        plt.plot(t, approx_sol_euler(yp, t, y0), label=f"Euler h=L/{div*10}",
c=col_e, lw=1)
        plt.plot(t, approx_sol_heun(yp, t, y0), label=f"Heun h=L/{div*10}", c=col_h,
lw=1)

def plot_sol(y: Callable[f64, f64], t0: f64, L: f64):
    t = np.linspace(t0, t0 + L)
    plt.plot(t, y(t), label="Actual", c="black", lw=1)

def plot_error(yp: Callable[f64, f64], y: Callable[f64, f64], t0: f64, L: f64, y0:
f64, divrange: tuple(int, int), local: bool):
    divs = np.arange(divrange[0], divrange[1] + 1)
```

```

euler = np.zeros(divs.size, dtype=f64)
heun = np.zeros(divs.size, dtype=f64)
for i, div in enumerate(divs):
    t = np.linspace(t0, t0 + (L/2 if local else L), div * (5 if local else 10) +
1)
    euler[i] = approx_sol_euler(yp, t, y0)[-1]
    heun[i] = approx_sol_heun(yp, t, y0)[-1]
y_acc = y(t0 + (L/2 if local else L))
euler = np.abs(euler - y_acc)
heun = np.abs(heun - y_acc)
h = L / (10 * divs)
plt.plot(h, euler, label="Euler", c="red")
plt.plot(h, heun, label="Heun", c="blue")

def plot_ODE(yp: Callable[[f64, f64], f64], y: Callable[[f64, f64], f64], t0: f64, L: f64, y0: f64,
title: str, ylog: bool = False):
    approx_sol(yp, t0, L, y0)
    plot_sol(y, t0, L)
    plt.legend()
    if ylog:
        plt.yscale("log")
    plt.title(title)
    plt.xlabel("t")
    plt.ylabel("y")
    plt.show()

    plot_error(yp, y, t0, L, y0, (1, 100), True)
    plt.xscale("log")
    plt.yscale("log")
    plt.legend()
    plt.title(f"{title} - Local Error @ {t0 + L/2:.2f}")
    plt.xlabel("h")
    plt.ylabel("Absolute Error")
    plt.show()

    plot_error(yp, y, t0, L, y0, (1, 100), False)
    plt.xscale("log")
    plt.yscale("log")
    plt.legend()
    plt.title(f"{title} - Global Error @ {t0 + L:.2f}")
    plt.xlabel("h")
    plt.ylabel("Absolute Error")
    plt.show()

```

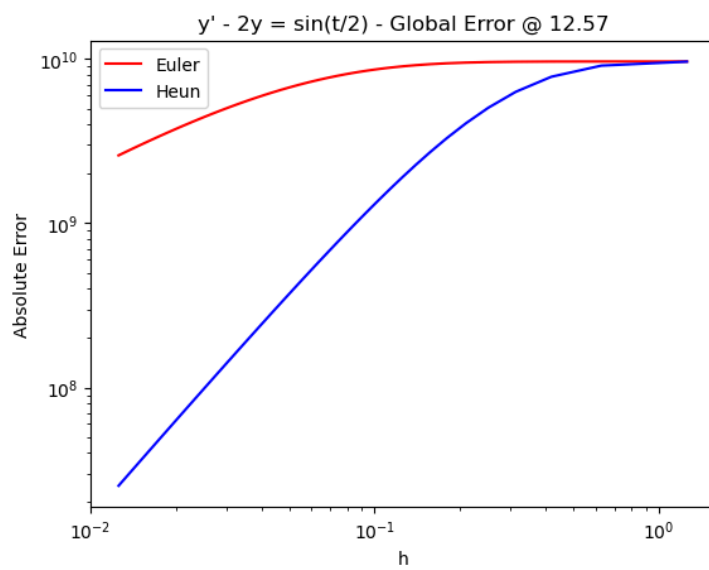
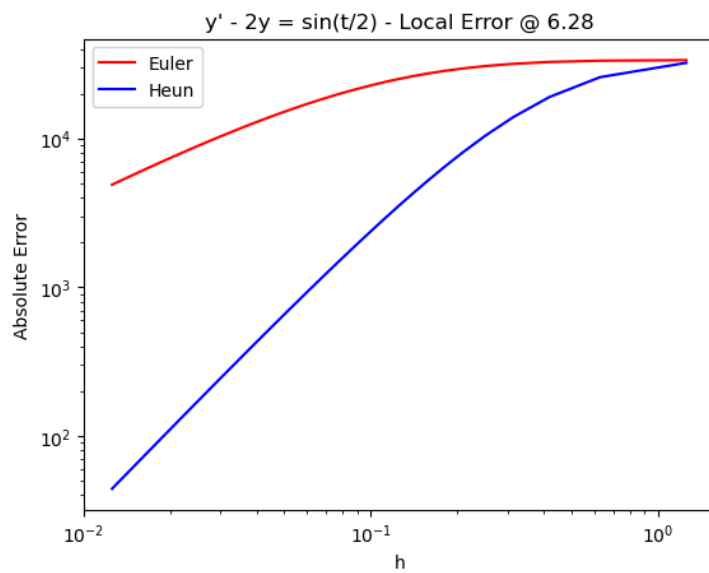
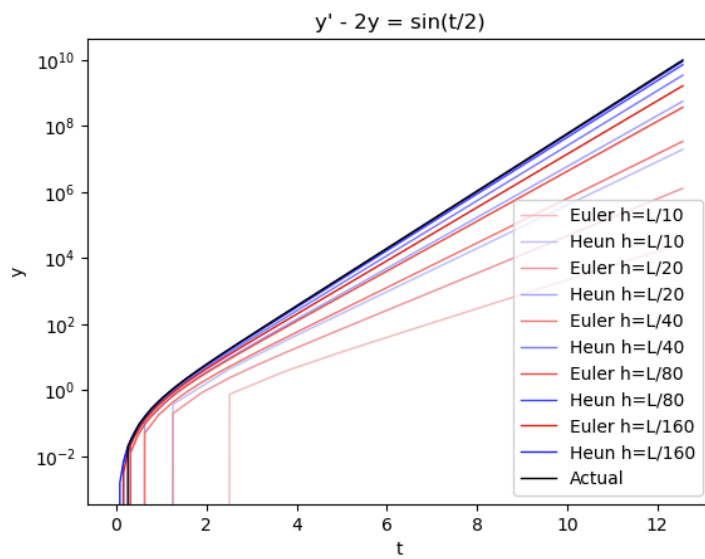
Results

ODE 4

```

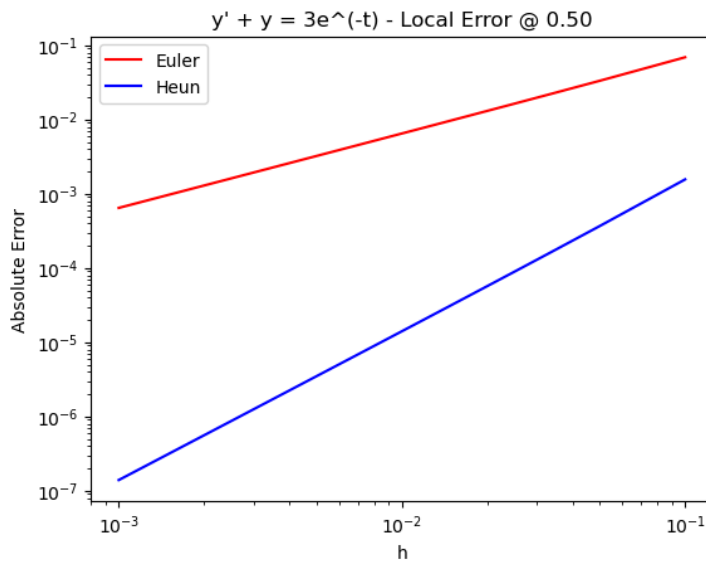
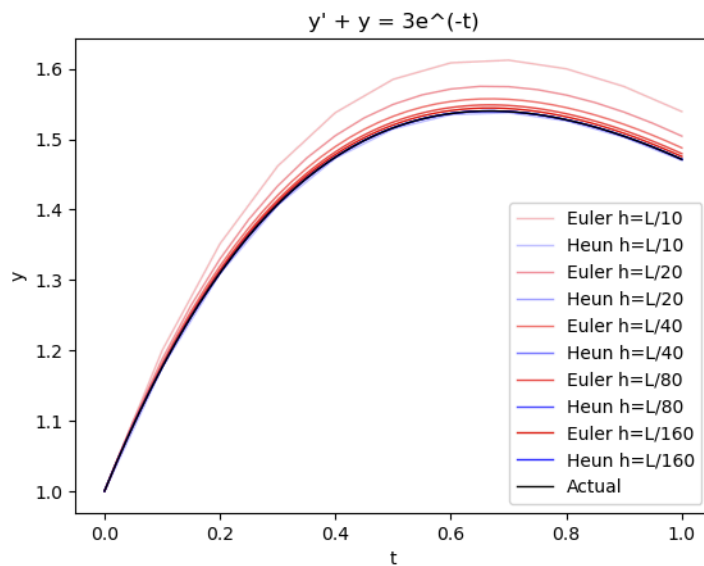
plot_ODE(
    yp=lambda t, y: np.sin(t/2) + 2 * y,
    y=lambda t: (2/17)*np.exp(2*t) - (2/17)*np.cos(t/2) - (8/17)*np.sin(t/2),
    t0=0,
    L=4*np.pi,
    y0=0,
    title="y' - 2y = sin(t/2)",
    ylog=True
)

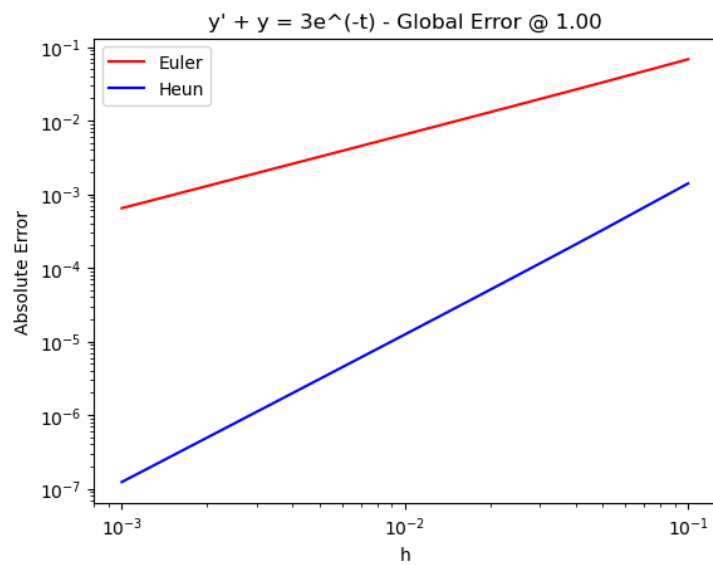
```



ODE 8

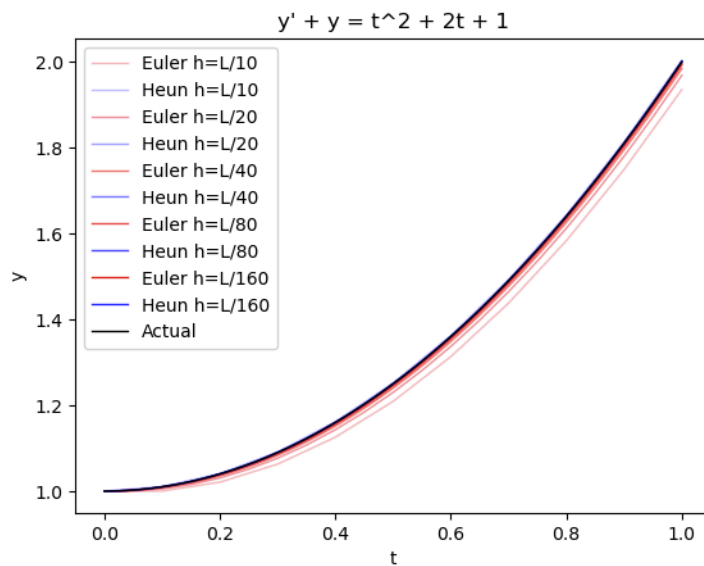
```
plot_ODE(  
    yp=lambda t, y: 3*np.exp(-t) - y,  
    y=lambda t: np.exp(-t) + 3*t*np.exp(-t),  
    t0=0,  
    L=1,  
    y0=1,  
    title="y' + y = 3e^(-t)",  
)
```

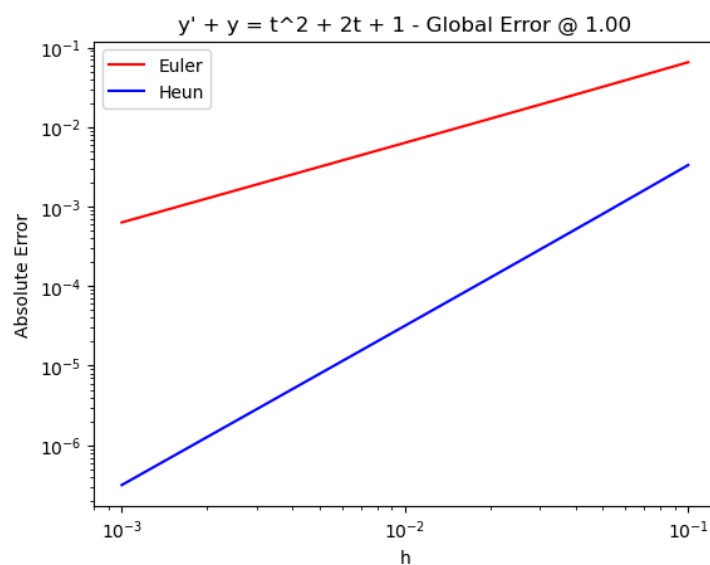
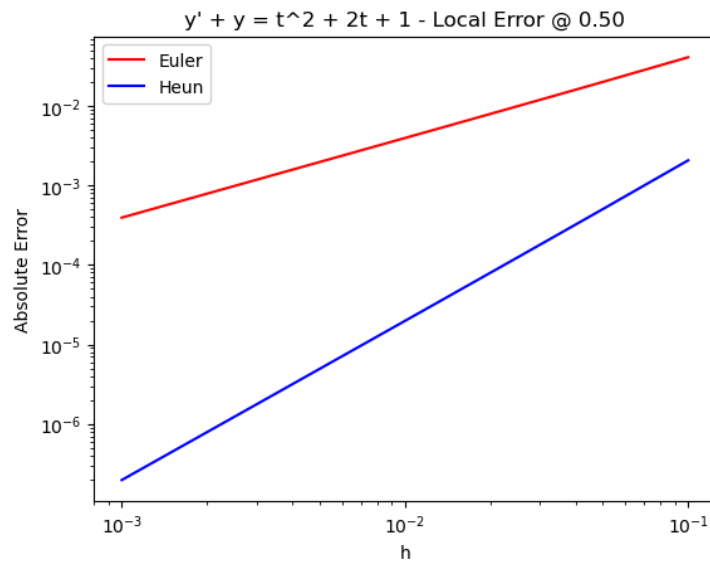




ODE 12

```
plot_ODE(
    yp=lambda t, y: t*t + 2*t + 1 - y,
    y=lambda t: t*t + 1,
    t0=0,
    L=1,
    y0=1,
    title="y' + y = t^2 + 2t + 1",
)
```





Discussion

We can see from the error plots that Heun's method converges faster than Euler's. Euler's method did indeed scale with $O(h)$ while Heun's method's error scaled with $O(h^2)$.

References

- <https://numpy.org/doc/stable/index.html>
- <https://matplotlib.org/stable/index.html>
- <https://typing.python.org/en/latest/spec/callables.html>